

LEVEL 6

Chapter 7: Messaging & Event-Driven Architecture

System Design Master Roadmap — 9 Years of Experience Edition

Compiled August 2026



Chapter 7: Messaging & Event-Driven Architecture

← [Chapter 6: Distributed Systems](#) · [Index](#) · Next → [Chapter 8: Microservices](#)

You already operate Kafka in production, so this chapter is written to convert operational familiarity into interview-ready articulation — the "why," precisely stated, not just the "how."

7.1 Message Queue vs. Pub/Sub — the core distinction

Message Queue (point-to-point): a message is delivered to (and consumed by) exactly **one** consumer, even if multiple consumers are listening — once consumed, it's removed. Classic use: distributing work items across a pool of workers, where each job should be done once (e.g., "resize this image").

Pub/Sub (publish-subscribe): a message published to a topic is delivered to **every** subscriber independently — each subscriber gets its own copy. Classic use: broadcasting an event that multiple, independent parts of the system each need to react to differently (e.g., "order placed" → inventory service reserves stock, notification service sends a confirmation email, analytics service logs the event — all three should see it, each doing something different).

Why this distinction is the first thing to nail in an interview: picking the wrong one is a subtle but real correctness bug — using a plain queue when you actually need three independent services to each process every event means only one of them will ever see it (whichever consumer happens to grab it first), silently breaking the other two.

7.2 Kafka — Deep Dive

Kafka is really a **distributed, partitioned, replicated commit log** — not a traditional message queue — and that framing explains almost every one of its design decisions.

Concept	What it is
Producer	A client that writes (appends) messages to a topic
Consumer	A client that reads messages from a topic, tracking its own position

Concept	What it is
Topic	A named stream of messages — the logical channel producers write to and consumers read from
Partition	A topic is split into ordered, append-only partitions — each partition is what actually gets distributed across brokers and gives Kafka its horizontal scalability. Order is only guaranteed within a single partition , never across partitions of the same topic
Offset	Each message's position within its partition — a simple, ever-increasing integer. Consumers track "the last offset I've processed" to know where to resume
Consumer Group	A set of consumers that split a topic's partitions between them — each partition is consumed by exactly one consumer <i>within</i> a group at a time, which is how Kafka achieves both pub/sub (across different groups) and load-balanced work distribution (within one group) simultaneously
Replication	Each partition is replicated across multiple brokers (typically 3) for durability
Leader / Follower	Each partition has one broker as its leader (handles all reads/writes for that partition) and the rest as followers (replicate the leader's data, ready to take over if the leader fails)

Ordering: guaranteed only within a partition. To guarantee two related events are processed in order (e.g., two updates to the same order), you must route them to the *same* partition — done by producing with the same **partition key** (e.g., `order_id`), since Kafka hashes the key to consistently pick the same partition for that key.

Retention: unlike a traditional queue, Kafka doesn't delete a message once it's consumed — messages stay for a configured retention period (time-based or size-based), and multiple independent consumer groups can each read the same data at their own pace, even replay from an earlier offset. This is a genuinely different mental model from RabbitMQ/SQS and worth stating explicitly when comparing them.

Consumer group rebalancing: when a consumer joins or leaves a group (scaling up/down, or a crash), Kafka redistributes partitions among the remaining/new consumers. During a rebalance, consumption pauses briefly for the affected partitions — a real, known source of latency spikes in Kafka-based pipelines, and worth mentioning if asked about Kafka's operational quirks (you'll have seen this yourself in production).

Delivery semantics in Kafka: at-least-once by default (a producer retry after a lost ack can create a duplicate); Kafka supports idempotent producers (deduplicates retries at the partition level via sequence numbers) and transactions (atomic writes across partitions/topics for consume-process-produce pipelines) which together give effectively-exactly-once *within* Kafka — see Chapter 6.7 for the full nuance on why this doesn't extend to external side effects.

Dead Letter Queue (DLQ): when a consumer repeatedly fails to process a message (a poison pill — malformed data, a bug triggered only by this specific message), retrying forever blocks the partition indefinitely (since Kafka consumption is sequential per

partition — you can't skip ahead). The standard fix: after N failed attempts, move the message to a separate DLQ topic and move on, so one bad message doesn't halt the entire pipeline. The DLQ is then inspected/reprocessed manually or via tooling.

Retry topics: rather than retrying immediately inline (blocking the partition while you wait), route failed messages to a dedicated retry topic (sometimes several, with increasing delay — `retry-5s`, `retry-1m`, `retry-10m`) and have a separate consumer process those on a delay, feeding back into the main flow or eventually the DLQ. This pattern (popularized by Uber's and others' engineering blogs) keeps the main consumer's throughput unaffected by retry backoff delays.

Backpressure in Kafka: because Kafka decouples producers from consumers via the log, a slow consumer doesn't block the producer at all — messages simply accumulate in the topic (bounded by retention), and **consumer lag** (how far behind the consumer's offset is from the latest written offset) becomes your visibility metric. This is one of Kafka's biggest structural advantages over a synchronous call chain: the queue itself *absorbs* backpressure, buying you time to scale consumers out, rather than propagating the slowdown instantly to the producer.

7.3 RabbitMQ

A traditional **message broker** implementing AMQP, built around **exchanges** (which receive messages from producers and route them to queues based on rules — direct, topic, fanout, headers) and **queues** (where messages actually sit until consumed). Unlike Kafka, a message is typically removed once acknowledged/consumed — it's not a persistent replayable log by default.

Where RabbitMQ genuinely fits better than Kafka: complex routing logic (RabbitMQ's exchange types give you flexible routing patterns — route by topic wildcard, fan out to many queues, route by header — more naturally than Kafka's simpler topic-partition model), lower absolute throughput needs where operational simplicity matters more than Kafka's horizontal scale, and use cases needing per-message priority or more fine-grained per-message acknowledgment/requeue semantics.

7.4 AWS SQS, SNS, Kinesis

Service	Type	Key traits
SQS	Managed message queue	Point-to-point, fully managed (no infrastructure to run), Standard queues (at-least-once, best-effort ordering, very high throughput) vs. FIFO queues (exactly-once processing, strict ordering, lower throughput ceiling)

Service	Type	Key traits
SNS	Managed pub/sub	Fan-out to multiple subscribers (which can themselves be SQS queues, Lambda functions, HTTP endpoints, email/SMS) — the classic "SNS fans out to multiple SQS queues" pattern combines SNS's pub/sub with SQS's durable, poll-based consumption per subscriber
Kinesis Data Streams	Managed, partitioned log (AWS's closer analog to Kafka)	Ordered within a shard, replayable within a retention window, multiple independent consumers — reach for this over SQS/SNS when you specifically need Kafka-like stream semantics (replay, ordered partitions) without operating Kafka yourself

7.5 Choosing Between Them

Requirement	Choose	Why
High-throughput event streaming, need replay, multiple independent consumer groups	Kafka (or Kinesis if you want it fully managed on AWS)	Log-based, retains data, natively supports many independent readers at different paces
Simple task/job queue, distribute work across workers, don't need replay	SQS (or RabbitMQ if self-hosting/on-prem)	Simplest operational model for pure point-to-point work distribution
Complex routing rules (topic wildcards, conditional fan-out)	RabbitMQ	Exchange types built exactly for this
Fan-out one event to many independent downstream systems, fully managed	SNS (often SNS → multiple SQS queues)	Purpose-built pub/sub fan-out on AWS
Need strict per-entity ordering plus replay, fully managed on AWS	Kinesis	Ordered shards + retention, no infrastructure to run
Very low-latency, in-memory, simple pub/sub, already have Redis in the stack	Redis Streams / Redis Pub-Sub	No new infrastructure if Redis is already present; note plain Redis Pub/Sub has no persistence — a subscriber that's briefly disconnected loses messages, so use Redis Streams specifically (not plain Pub/Sub) if you need at-least-once delivery with a consumer group model similar to Kafka's, at smaller scale

Interview question: "You're designing a notification system that must fan out one 'order placed' event to inventory, email, SMS, and analytics services. Kafka or SQS?"

Ideal senior answer: "Kafka (or SNS fan-out to SQS if I want each consumer's queue to be independently manageable on AWS without running Kafka myself). The key requirement is that four independent consumers each need to see every event and process it at their own pace — that's a pub/sub shape, not a point-to-point queue. With Kafka specifically, each service just needs its own consumer group reading the same topic; if the SMS service goes down for an hour, it resumes from its last committed offset with nothing lost, which a plain point-to-point queue wouldn't give me as naturally once a message is already consumed and removed."

Chapter 7 Interview Drill

1. Explain precisely why Kafka only guarantees ordering within a partition, and how you'd guarantee two related events are processed in order.
2. What happens during a Kafka consumer group rebalance, and why does it matter operationally?
3. Walk through the DLQ + retry-topic pattern for a consumer that occasionally fails on a malformed message.
4. When would you choose RabbitMQ over Kafka, concretely?
5. Explain the difference between SQS Standard and FIFO, and when each is the right choice.

Next → [Chapter 8: Microservices](#) — service boundaries, communication patterns, and the failure modes unique to distributed service architectures.